EMD Support — Technical Specification

Electronic Miscellaneous Documents (EMDs) give every ancillary sold on a booking its own ticket number, so a post-booking change can reissue a single ancillary instead of cancelling the PNR and rebooking it. This specification covers the nine NDC suppliers that return `emdTickets` in the Book and Retrieve responses, the fields they send, and the schema and flow proposed to store and surface them.

1. Scope and supported suppliers

EMD support is supplier-specific: nine NDC suppliers return `emdTickets`, and the field is absent for every other supplier. The feature is read-only in this phase — the system stores and displays EMD numbers; it does not issue, void or reissue them.

In scope

- Parsing `emdTickets` from the Book and Retrieve Booking responses.
- Persisting EMD data in a new schema, linked to the existing booking, passenger and segment records.
- Displaying EMD numbers in PNR Query, E-Ticket and Itinerary.

Out of scope (this phase)

- Issuing, voiding or refunding an EMD.
- Executing the reissue itself. This feature is the enabler: it identifies which document to change. The reissue flow is a separate deliverable.
- Seats. Suppliers may return an EMD for a seat, but no supported supplier does so today; the model below handles it without change if one starts.

Supported suppliers

Supplier (business name)	`Supplier` enum	Connector name
Riyadh Air	`RiyadhAir`	`RiyadhAir`
Tarco	`Tarco`	`Tarco`
Amadeus NDC	`AmadeusNDC`	`AmadeusNDC`
Aegean	`AegeanV21`	`AegeanNdc`
Travelport NDC	`TravelportNdc`	`TravelportNdc`
Galileo	`Galileo`	`NDCGal`
Gulf Air	`GulfAir`	`GulfAir`
Egypt Air	`EgyptAir`	`EgyptAirEgy`
Air Cairo	`AirCairo`	`AirSMCairo`

A supplier outside this list must behave exactly as it does today. `emdTickets` is optional in the contract, so the absence of the node is a normal response, never an error.

2. EMD granularity rules

An EMD is issued per ancillary, not per booking and not per ancillary basket. These four rules drive the whole design.

1. **One EMD per ancillary item.** If a passenger buys a meal, a seat and a bag, the supplier issues a separate EMD for each. There is no single document covering the selection.
2. **Coverage is partial and supplier-driven.** A supplier may return an EMD for the bag and none for the seat. A missing EMD is not an error and must not block the booking.
3. **EMDs belong to a passenger and link back to that passenger's flight ticket.** `connectedTicketNumber` carries the e-ticket number the EMD is attached to.
4. **The level of an EMD is the supplier's choice.** Most suppliers issue one EMD per journey; Riyadh Air issues one per segment for the same purchased ancillary (see section 7).

The unit of reissue therefore is one EMD. Changing a bag after ticketing means reissuing the bag's EMD only — the flight ticket and every other ancillary stay untouched. Without the EMD number the change requires cancelling and rebuilding the booking, which is the cost this feature removes.

3. Integration contract

Integration adds one optional node, `emdTickets`, to the Book and Retrieve Booking responses. It is a map keyed by passenger reference (`T1`, `T2`, … — the same keys already used by `tickets`, `passengersList` and `passengersSelectedAncillaries`), whose value is a list of EMDs for that passenger.

```
emdTickets: Map<String, List<EmdTicketType>>
                └─ passenger ref id
```

`EmdTicketType`

Field	Type	Null?	Meaning
`emdTicketNumber`	String	No	The EMD document number. The value shown to users and used for reissue.
`connectedTicketNumber`	String	No	The passenger e-ticket number this EMD is attached to. Matches a `tickets[paxRef].ticketNumber`.
`issueDate`	Date	No	Date the EMD was issued.
`ticketStatus`	`TicketStatusEnum`	No	Reuses the existing enum (`Booked`, …). No new values expected.
`ancillaryCode`	String	No	The ancillary this EMD pays for. Matches `passengersSelectedAncillaries[paxRef]…ancillaryCode`.
`segmentRefIds`	List<String>	No	Segments the EMD applies to. One entry for a per-segment EMD, several for a journey-level EMD.
`emdCancelationRules`	List< `EmdCancelationRuleType` >	Yes	Cancellation penalties by journey stage. Empty or absent when the supplier sends none.

`EmdCancelationRuleType`

Field	Type	Null?	Meaning
`description`	String	No	Supplier free text, shown as-is.
`journeyStageCode`	Enum	No	`BeforeDeparture`, `NoShow`, `AfterDeparture`.
`penality`	Money (`amount`, `currency`)	No	Penalty charged at that stage. Note the supplier's spelling of the field.

Worked example — Riyadh Air, PNR `1LPK5A`

One passenger (`T1`) bought one ancillary, `C0C1` (Checked Bag 15kg) at 151.00 SAR for a CAI–RUH–JED journey. The supplier returned **two** EMDs for that single purchase, one per segment:

EMD number	Connected ticket	Ancillary	Segment
`1224260082313`	`1222160135367`	`C0C1`	`DMS3b98e872…` (CAI–RUH)
`1224260082314`	`1222160135367`	`C0C1`	`DMSac2063c2…` (RUH–JED)

This is the case section 7 handles.

4. Proposed solution — data model

EMD data does not fit any existing table: `TT_TS_FB_PAX_SEG_BOOK_DETAIL` holds one ticket per passenger per segment and has no ancillary dimension, and `TT_TS_FB_PASSENGER_SSR` holds the purchased ancillary but no document number. The proposal is one new table that stores each passenger's EMDs as a single JSON value.

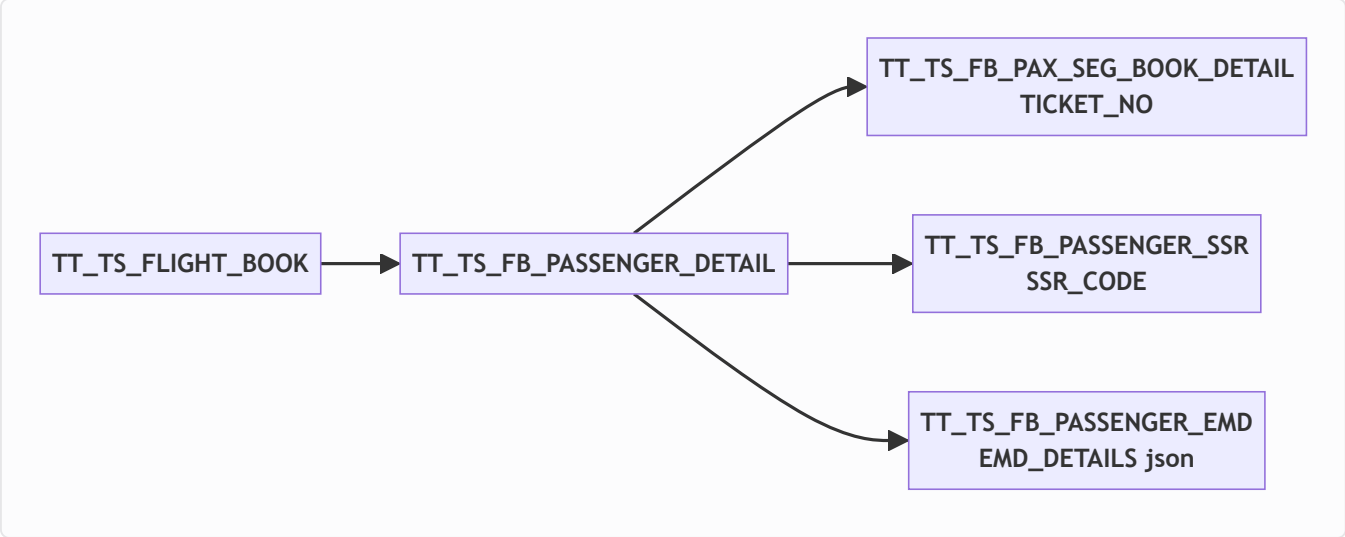
The supplier already keys `emdTickets` by passenger reference (`T1`, `T2`, …), and everything under that key — the EMD list, the segment references, the cancellation rules — is only ever read as a unit for that passenger. So the record is one row per passenger, and the payload is the supplier's own structure kept whole.

`TT_TS_FB_PASSENGER_EMD` — one row per passenger that has EMDs.

Column	Type	Key	Notes
`ID`	number	PK, FK → `TT_TS_FLIGHT_BOOK`	Same join the SSR table uses.
`FB_BOOKING_REF_NO`	varchar	PK	Booking reference.
`PASSENGER_NO`	number	PK	Resolved from the supplier passenger ref.
`PAX_REF_ID`	varchar(10)		The supplier key as received (`T1`). Kept so a response can be traced back without re-matching.
`EMD_DETAILS`	JSON		The `emdTickets[paxRef]` array, stored verbatim.
`SUPPLIER_NAME`	varchar		Denormalised for reporting, as elsewhere in the booking schema.
`CREATION_TIME`	timestamp		
`LAST_MOD_TIME`	timestamp		Updated when Retrieve Booking refreshes the row.

`EMD_DETAILS` holds the array exactly as section 3 defines it — `emdTicketNumber`, `connectedTicketNumber`, `issueDate`, `ticketStatus`, `ancillaryCode`, `segmentRefIds` and `emdCancelationRules` per entry. Nothing is flattened or dropped on write.

How it joins



The links from an EMD back to the e-ticket and to the ancillary live inside the JSON, not in the schema: `connectedTicketNumber` matches `TT_TS_FB_PAX_SEG_BOOK_DETAIL.TICKET_NO` and `ancillaryCode` matches `TT_TS_FB_PASSENGER_SSR.SSR_CODE`. Both are resolved in Java when the read model is built (section 5), not by SQL.

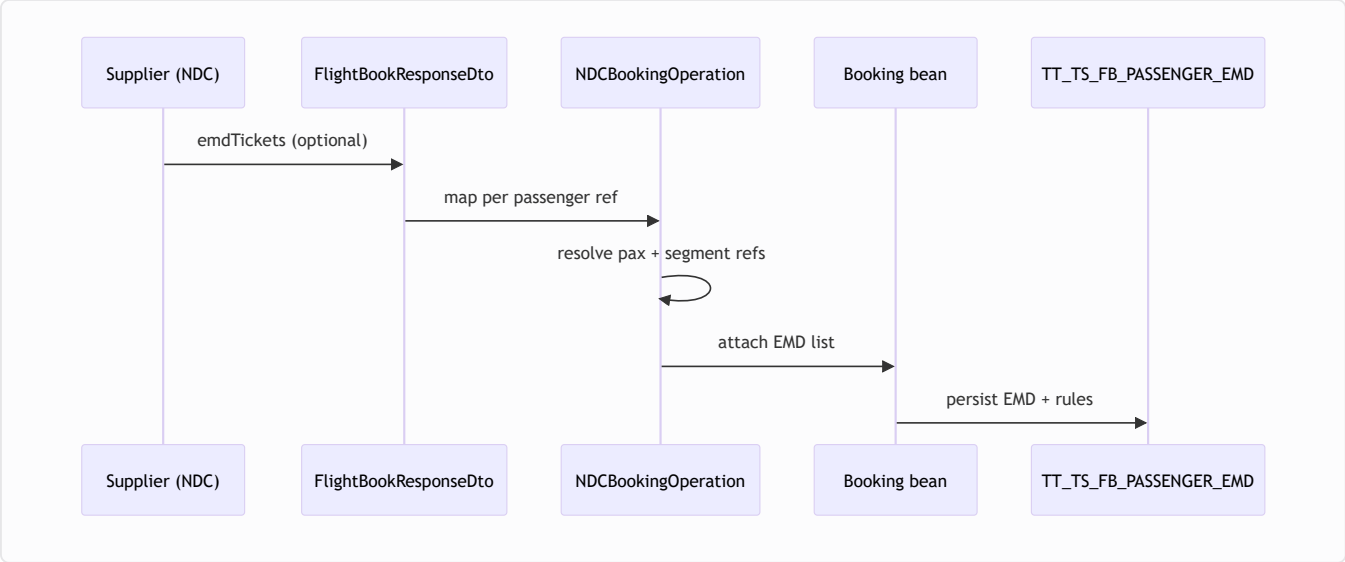
What this buys, and what it costs

One table instead of two	Cancellation rules are a repeating group inside a repeating group. As JSON they need no child table and no sequence columns.
No DDL when the contract grows	A field Integration adds later is stored the day it arrives.
Nothing to reconcile on write	Riyadh Air's two EMDs for one bag are simply two array entries. No per-segment row resolution at write time.
Lookup by EMD number is not indexed	Support searching for a single document number needs a JSON path predicate or a full scan. If that becomes a real query, add a generated column on `emdTicketNumber` and index it — no data migration required.
No referential integrity on the payload	`ancillaryCode` and `segmentRefIds` are not enforced by the database. Validation happens at read time and unresolved values render as plain text.
Reporting reads JSON	Any future report over EMDs parses the column rather than joining rows.

Column type — the native `JSON` type needs MySQL 5.7 or later; the configured dialect is `MySQL5InnoDBDialect`, which does not tell us the server version. Confirm it before the DDL. On an older server the column is `LONGTEXT` and the application behaves identically, losing only server-side JSON predicates.

5. Proposed solution — flow

EMDs follow the path e-tickets already follow. `FlightBookResponseDto` gains one field, the NDC booking operation maps it the way `defineTickets` maps `tickets`, and the booking save writes the new tables.



Step 1 — contract

Add `Map<String, List<EmdTicketType>> emdTickets` to `FlightBookResponseDto` and to `RetrieveBookingResponseDto`, with `EmdTicketType` and `EmdCancelationRuleType` as new shared types. Both DTOs already carry passenger-keyed maps, so this is additive and breaks nothing.

Step 2 — mapping

Add `defineEmdTickets(…)` in the NDC booking operation, called next to `defineTickets(…)` and guarded the same way (return immediately when `emdTickets` is null or empty). It resolves one reference only — the passenger — using the `passengersClone` matching already used by `defineTickets`: supplier key `T1` becomes `PASSENGER_NO`. The EMD list for that passenger is then serialised whole into `EMD_DETAILS`. Segment and ancillary references stay inside the payload and are resolved at read time (step 4).

Step 3 — persistence

Extend the booking-save path to write one `TT_TS_FB_PASSENGER_EMD` row per passenger in the same transaction as the rest of the booking. Retrieve Booking follows the same mapping and overwrites `EMD_DETAILS` on the existing row, so a booking imported or re-synced later picks EMDs up and a status change is not lost.

Step 4 — read model

One query object loads the EMD rows for a booking, deserialises `EMD_DETAILS` back into `List<EmdTicketType>`, and does the resolution the schema no longer does:

Payload value	Resolved against	Result
`ancillaryCode`	`TT_TS_FB_PASSENGER_SSR.SSR_CODE` for that passenger	The ancillary line the EMD is rendered on.
`connectedTicketNumber`	`TT_TS_FB_PAX_SEG_BOOK_DETAIL.TICKET_NO`	The passenger e-ticket the EMD is attached to.
`segmentRefIds`	The booking's segment records	Segment order, used for the grouping in section 7.

The display modules in section 6 consume that object; none of them read `EMD_DETAILS` directly.

Failure handling

Because the payload is stored verbatim, a write can only fail on the passenger match. If a supplier passenger key cannot be matched, the EMDs are logged and skipped rather than failing the booking: the money has moved, and a display gap is recoverable while a failed save is not.

At read time, a value that does not resolve — an `ancillaryCode` with no matching SSR row, or a `segmentRefId` not in the booking — is logged and the EMD renders unattached rather than disappearing.

6. Display rules

The system stores every field in section 3 but displays only two: `emdTicketNumber` and `connectedTicketNumber`. The rest is held for the reissue flow and for support investigations.

Module	Shows EMD	Placement
PNR Query	Yes	Beside the ancillary it belongs to.
E-Ticket	Yes	Beside the ancillary it belongs to.
Itinerary	Yes	Beside the ancillary it belongs to.

Rules that apply to all three:

- An EMD is rendered against its ancillary line, never as a standalone list. The agent reads “Checked Bag 15kg — EMD 1224260082313”, not a table of numbers.
- An ancillary with no EMD renders exactly as it does today. No placeholder, no empty label.
- A booking from a non-supported supplier is visually unchanged.
- Numbers are shown as returned, with no reformatting or truncation.

7. Riyadh Air — per-segment EMDs

Riyadh Air treats baggage as a per-segment product. One bag bought for a two-segment journey returns two EMDs with the same `ancillaryCode` and the same `connectedTicketNumber`, differing only in `segmentRefIds`. Every other supported supplier returns one EMD for the journey.

The system sells that bag as a single journey-level ancillary at a single price, so the two EMDs must be presented as one line. The rule agreed with Integration:

> Keep both EMDs as separate entries in the passenger's `EMD_DETAILS` array, each with its own `segmentRefIds`. When building the read model, group the entries by (`connectedTicketNumber` , `ancillaryCode`) and display each group as one ancillary line whose EMD numbers are concatenated in segment order.

For the example in section 3, the agent sees one line — *Checked Bag 15kg — EMD 1224260082313, 1224260082314* — at 151.00 SAR.

Concatenating is safe here because the price is fixed per segment and the ancillary is identical across them; no arithmetic is performed on the group. Two points follow:

- Storage stays per-EMD inside the payload. Grouping happens in the read model only, so a future reissue can still address one segment's document.
- The grouping key is generic, so a supplier that returns one EMD per journey produces a group of one and needs no special case.